

Virtual Personal Finance Advisor

CUPRINS

LISTA DE FIGURI	5
LISTA DE SECVENTE DE COD	7
LISTA DE TABELE	7
LISTA DE ABREVIERI	9
1 INTRODUCERE	11
1.1 Contextul și Actualitatea Temei	11
1.2 Motivarea Lucrării	11
1.3 Obiectivele Propuse	11
2 STADIUL ACTUAL ÎN DOMENIUL ASISTENȚILOR FINANCIARI	13
2.1 Analiza Soluțiilor Existente	13
2.2 Tehnologii și Algoritmi Utilizați în FinTech	13
2.3 APIs și Surse de Date Financiare	15
3 ANALIZA CERINȚELOR ȘI PROIECTAREA SISTEMULUI	17
3.1 Specificarea Cerințelor	17
3.2 Arhitectura Sistemului	18
3.3 Proiectarea Fluxului de Date	20
4 IMPLEMENTAREA ALGORIMILOR DE DETECȚIE ȘI PREDICȚIE	23
4.1 Ansamblul Hibrid pentru Detecția Anomaliilor	23
4.2 Mecanismul de Voting	23
4.3 Modele de Predicție a Sănătății Portofoliului	24
5 IMPLEMENTAREA SOFTWARE A APLICAȚIEI	25
5.1 Tehnologii Utilizate	25
5.2 Detalii de Implementare	26
5.3 Interfața Utilizatorului	32
6 TESTARE, VALIDARE ȘI ANALIZA DEFECTELOR	35
6.1 Strategia de Testare	35
6.2 Analiza și Documentarea Defectelor	35
6.3 Evaluarea Rezilienței	37
7 MANUAL DE UTILIZARE	39
8 CONCLUZII ȘI CONTRIBUȚII PERSONALE	41
8.1 Concluzii	41

8.2	Contribuții Personale	41
8.3	Direcții Viitoare de Dezvoltare	42

LISTA DE FIGURI

2.1	Principiul învățării prin ansamblu pentru detecția anomaliilor	14
3.1	Diagrama cazurilor de utilizare	17
3.2	Ciclul de viață al unei cereri prin straturile aplicației	21
5.1	Palieretele tehnologice ale aplicației	25
5.2	Diagrama entitate-relație a modelului de date	27
5.3	Fluxul de autorizare OAuth2 PKCE (BT PSD2)	29
5.4	Pipeline-ul de procesare a tranzacțiilor bancare	30
5.5	Harta de navigare (IndexedStack cu cinci file)	33
5.6	Ecranul de ansamblu cu harta treemap a portofoliului	34

LISTA DE TABELE

2.1	Comparație între Virtual Personal Finance Advisor și Soluții Existente	13
2.2	Clase de Abordări pentru Detecția Anomaliilor în Finanțe	14
2.3	Paradigme de interacțiune cu modele large de limbaj	14
2.4	Surse de date externe utilizate	15
2.5	Endpoint-uri eToro utilizate	15
2.6	Endpoint-uri BT PSD2 (AISP) utilizate	16
3.1	Cerințe funcționale	17
3.2	Cerințe non-funcționale	18
3.3	Straturile aplicației și responsabilitățile lor	20
5.1	Dependențe principale (backend), versiunile și rolurile lor	26
5.2	Routele aplicației	26
5.3	Entitățile modelului de date	27
5.4	Uneltele MCP disponibile agentului Tori	28
5.5	Conturi-exemplu folosite ca rezervă în sandbox	30
5.6	Extras din taxonomia de categorii	30
5.7	Mecanisme de securitate	32
5.8	Filele principale ale aplicației	33
6.1	Defecte, Mecanisme de Implementare și Soluții de Toleranță	36
6.2	Defecte, Clasificare și Impact	36
8.1	Gradul de îndeplinire a obiectivelor	41
8.2	Contribuții personale și componentele asociate	42

LISTA DE ABREVIERI

API	Application Programming Interface
AISP	Account Information Service Provider
AES	Advanced Encryption Standard
ASGI	Asynchronous Server Gateway Interface
CBC	Cipher Block Chaining
JWT	JSON Web Token
LLM	Large Language Model
LRU	Least Recently Used
LSTM	Long Short-Term Memory
MAPE	Mean Absolute Percentage Error
MCP	Model Context Protocol
ML	Machine Learning
MSE	Mean Squared Error
OHLCV	Open-High-Low-Close-Volume
ORM	Object-Relational Mapping
PCA	Principal Component Analysis
PKCE	Proof Key for Code Exchange
PSD2	Payment Services Directive 2
RBF	Radial Basis Function
ReAct	Reasoning + Acting
REST	Representational State Transfer
SVD	Singular Value Decomposition
SVM	Support Vector Machine
TTL	Time To Live
UI	User Interface
WSGI	Web Server Gateway Interface

1 INTRODUCERE

Anul 2025 marchează o importantă tranziție de la API-uri cu acces limitat la API-uri publice, deschise, datorită așa-numitei paradigme "Builders' economy".

1.1 Contextul și Actualitatea Temei

Paradigma "Builders' economy" a permis democratizarea accesului la investiții către simplii utilizatori. Lansarea eToro Public API a avut o mare influență în ideea de creare a proiectului Virtual Personal Finance Advisor.

Proiectul propus, Virtual Personal Finance Advisor, își propune să ofere utilizatorilor o imagine mai clară asupra situației lor financiare. Acest asistent inteligent este capabil să analizeze și să prelucreze date despre investițiile utilizatorilor, oferind un raport detaliat în limbaj natural și conversațional.

1.2 Motivarea Lucrării

În era informațională, utilizatorii se confruntă adesea cu volume mari de date și informații financiare, care fac dificilă înțelegerea și gestionarea eficientă a activelor, mai ales când sunt în număr mare și influențate de o mulțime de factori. Principalele provocări includ:

- complexitatea analizei tehnice a datelor financiare;
- lipsa proactivității în identificarea anomaliilor sau risurilor;
- bariere tehnice în utilizarea diferitor infrastructuri de date și API-uri;

Astfel, aplicația se poziționează ca un pod între utilizatori și datele financiare de pe platforme, oferind o analiză în timp real bazată pe inteligența artificială.

1.3 Obiectivele Propuse

Obiectivul principal al acestui proiect este crearea unui asistent financiar personal de informare care să relice și să extindă funcționalitățile asistentului Tori de la eToro. Aplicația este destinată doar ca un instrument de informare, nu oferă sfaturi financiare sau recomandări de investiții, eliminând astfel riscurile legale asociate cu oferirea de consultanță financiară sau execuția tranzacțiilor, ce este ilegală în multe jurisdicții.

2 STADIUL ACTUAL ÎN DOMENIUL ASISTENȚILOR FINANCIARI

Piața aplicațiilor financiare personale a evoluat semnificativ în ultimii ani, oferind o varietate de soluții pentru gestionarea activelor utilizatorilor.

2.1 Analiza Soluțiilor Existente

În tabelul 2.1 Virtual Personal Finance Advisor este comparat cu alte aplicații financiare personale de pe piață, evaluând capacitățile de analiză și acces la date.

Table 2.1 - Comparație între Virtual Personal Finance Advisor și Soluții Existente

Caracteristică	eToro	Robinhood	Revolut	Virtual Personal Finance Advisor
Focus	Tranzacționare	Tranzacționare	Tranzacționare	Analiză și Informare
Acces API	Public API	Limitat	Limitat	MCP API
Asistență AI	Da	Nu	Da	Agentic (MCP) & ML
Analiză Sentiment	Da	Nu	Nu	Euristică

Referința principală a acestui proiect este asistentul Tori de la eToro, un agent conversațional bazat pe inteligența artificială care utilizează date publice, profiluri active și portofolii publice pentru a răspunde la întrebări de tipul "Compară NVIDIA cu Tesla". Spre deosebire de agentul de la eToro, care are limita de acces de doar 3 întrebări pe săptămână, Virtual Personal Finance Advisor oferă o arhitectură deschisă bazată pe Model Context Protocol, fiind capabil să proceseze un volum mare de întrebări și să ofere răspunsuri în timp real. La fel, spre deosebire de agentul de la eToro, Virtual Personal Finance Advisor poate interoga un număr mult mai mare de active financiare, oferind o acoperire mult mai largă a profilului de investiții al utilizatorului.

2.2 Tehnologii și Algoritmi Utilizați în FinTech

În ultimii ani, domeniul tehnologiei financiare (FinTech) a adoptat tehnologii de învățare automată, procesare a limbajului natural (NLP), și modele large de limbaj (LLM) pentru a îmbunătăți experiența utilizatorilor transformând volume mari de date financiare în informații utile. Lucrarea dată integrează detecția anomaliilor, agenți conversaționali bazați pe LLM, și vizualizarea financiară.

Pentru identificarea anomaliilor în datele financiare, adică a comportamentelor neobișnuite în portofoliul utilizatorului, se folosesc metode nesupervizate, deoarece datele etichetate sunt rare în domeniul financiar. În tabelul 2.2 comparăm cele trei mari clase de abordări.

Table 2.2 - Clase de Abordări pentru Detecția Anomaliilor în Finanțe

Abordare	Caracteristici	Exemple
Supervizată	Necesită date etichetate	Random Forest, Rețele neuronale clasificatoare
Nesupervizată	Nu necesită etichete	Isolation Forest, One-Class SVM, Autoencoder
Ansamblu	Combină mai multe module	Soft Voting, Bagging, Stacking

Această aplicație folosește abordarea de tip *Ensemble Learning*, combinând trei modele nesupervizate. Justificarea acestei alegeri se bazează pe faptul că erorile individuale ale fiecărui model sunt necorelate, iar media lor ponderată reduce rata de fals pozitive și fals negative, îmbunătățind astfel acuratețea detecției anomaliilor. Instantierea acestor modele, parametrii și formula de vot vor fi detaliate în Capitolul 4.

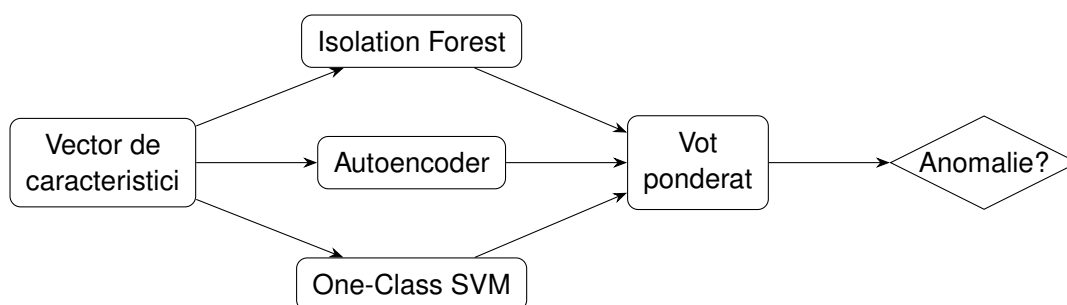


Figure 2.1 - Principiul învățării prin ansamblu pentru detecția anomaliilor

Agenții conversaționali în prezent sunt adesea bazați pe modele large de limbaj (LLM) și sunt capabili să raționeze și să folosească unelte externe pentru a răspunde la întrebări complexe. În tabelul 2.3 prezentăm principalele paradigme de utilizare a LLM-urilor.

Table 2.3 - Paradigme de interacțiune cu modele large de limbaj

Paradigmă	Descriere
Zero-shot / Few-shot	Modelul se bazează doar pe cunoștințele de la antrenare
RAG	Se introduc în prompt fragmente dintr-o bază de cunoștințe
Tool-calling	Datele se obțin prin apelarea uneltelor externe
Agentic (ReAct)	Datele se obțin prin interacțiunea modelului cu unelte externe, într-o buclă

Aplicația propusă folosește paradigma *ReAct* (*Reasoning & Acting*), în care agentul conversațional intercalează raționamentul cu apelurile uneltelor externe, reușind astfel să justifice răspunsurile cu date reale în timp real. Conectarea modelului la sursele de date se face prin intermediul *Model Context Protocol*, care expune funcții Python ca unelte apelabile de către agent.

2.3 APIs și Surse de Date Financiare

Aplicația integrează patru surse de date externe, fiecare având un rol specific în profilul financiar al utilizatorului, după cum se prezintă în tabelul 2.4.

Table 2.4 - Surse de date externe utilizate

Sursă	Tip date	Autentificare	Limitări
eToro Public API	Portofolii publice	Cheie API	Endpoint-uri publice
Yahoo Finance	Cotații, istoric	Fără cheie	Endpoint neoficial
BT PSD2 (AISP)	Conturi, tranzacții	OAuth2 + PKCE	Sandbox
Google Gemini	Limbaaj natural	Cheie API	Cotă per minut

Sursa principală a investițiilor utilizatorului este *eToro Public API*. Aplicația folosește endpoint-urile publice pentru a accesa portofoliile deschise ale utilizatorilor, fără a avea nevoie de autentificare. Răspunsurile de la eToro conțin informații despre alocația procentuală a activelor și profitul nerealizat, iar activele sunt reprezentate ca instrumente unice, de aceea avem nevoie să obținem și metadatele lor ca simbol, nume, și clasă. În tabelul 2.5 prezentăm endpoint-urile eToro utilizate în aplicație.

Table 2.5 - Endpoint-uri eToro utilizate

Endpoint	Rol
/api/v1/user-info/people/{user}/portfolio/live	Portofoliul public
/api/v1/market-data/instruments	Metadate instrumente (simbol, nume, clasă)

```
1 {  
2   "positions": [  
3     {"instrumentId": 1001, "investmentPct": 12.5, "netProfit": 8.3},  
4     {"instrumentId": 1245, "investmentPct": 7.0, "netProfit": -2.1}  
5   ]  
6 }
```

Listing 2.1 - Exemplu de răspuns (simplificat) al endpoint-ului de portofoliu

Cotațiile și istoricul de prețuri sunt obținute de la *Yahoo Finance*, accesat printr-un endpoint neoficial, fără cheie de acces sau autentificare, din biblioteca Python *yfinance*. Există și posibilitatea ca sursa să limiteze cererile temporar sau să devină indisponibilă, de aceea acest risc este tratat în aplicație prin mecanisme de reziliență descrise în Capitolul 6.

Datele bancare sunt obținute prin intermediul interfeței *PSD2* (standardul *NextGenPSD2 / Berlin-Group*), în calitate de *AISP* (*Account Information Service Provider*). Pentru a accesa datele conturilor și istoricul tranzacțiilor. Accesul se face printr-un proces de autorizare bazat pe *OAuth2* cu extensia *PKCE*, în urma consimțământului explicit al utilizatorului (Tabelul 2.6 prezintă endpoint-urile utilizate).

Table 2.6 - Endpoint-uri BT PSD2 (AISP) utilizate

Endpoint	Rol
POST /v2/consents	Creează consimțământul de acces
POST /oauth/token	Schimbă codul de autorizare pe token
GET /v2/accounts	Listează conturile
GET /v2/accounts/{id}/balances	Soldul unui cont
GET /v2/accounts/{id}/transactions	Tranzacțiile (booked)

În baza agentului conversațional, se folosește modelul *Google Gemini* (gemini-3.1-flash-lite), accesat prin biblioteca langchain-google-genai. Agentul folosește paradigma *ReAct* și apelează uneltele externe expuse prin serverul *MCP* (*Model Context Protocol*), descris în detaliu în Capitolul 5.

3 ANALIZA CERINȚELOR ȘI PROIECTAREA SISTEMULUI

În acest capitol sunt abordate întrebări legate de ce și cum trebuie să facă aplicația, descrierea modului în care sunt folosite componentele sale și fluxul de date între acestea.

3.1 Specificarea Cerințelor

Cerințele aplicației sunt grupate în două categorii: cerințe funcționale (ce trebuie să facă sistemul) și cerințe non-funcționale (cum trebuie să se comporte sistemul). Actorul principal este *utilizatorul* (investitorul de retail), iar actorii secundari sunt: sistemele externe (eToro API, Yahoo Finance, Banca Transilvania, Google Gemini).

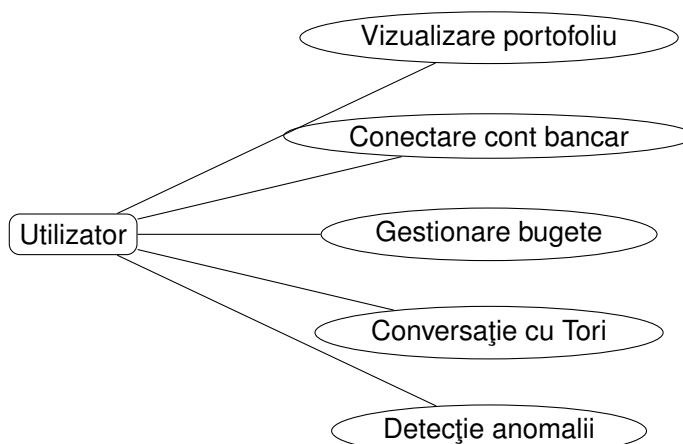


Figure 3.1 - Diagrama cazurilor de utilizare

În tabelul 3.1 sunt listate cerințele funcționale, fiecare având asociată un API endpoint care o implementează.

Table 3.1 - Cerințe funcționale

ID	Descriere	Endpoint
CF-01	Accesarea portofoliului de investiții	/etoro/portfolio
CF-02	Obținerea cotațiilor și a istoricului de preț	/market/...
CF-03	Conectarea la contul bancar și accesarea informațiilor	/bank/...
CF-04	Setarea bugetelor și monitorizarea lor	/budget/...
CF-05	Categorizarea tranzacțiilor	/expenses/...
CF-06	Detecția anomaliilor în portofoliu	/anomaly/analyze
CF-07	Agentul conversațional	/agent/chat
CF-08	Autentificarea și gestionarea sesiunii	/auth/...

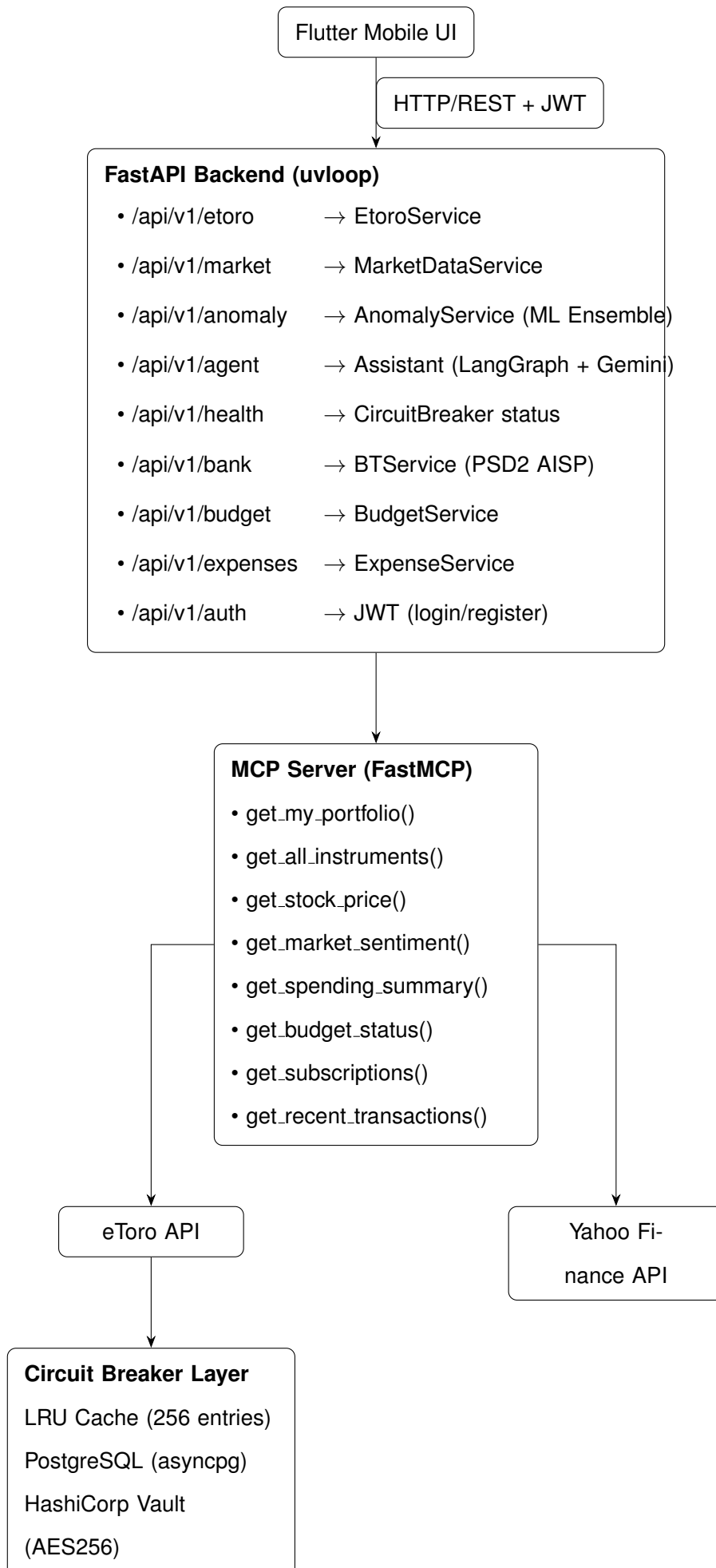
În tabelul 3.2 sunt listate cerințele non-funcționale, care descriu caracteristici de performanță, securitate și disponibilitate.

Table 3.2 - Cerințe non-funcționale

ID	Atribut	Descriere
CNF-01	Reziliență	Degradare grațioasă la eșecul unei surse externe
CNF-02	Securitate	Criptare AES-256, JWT, chei în HashiCorp Vault
CNF-03	Performanță	Procesare asincronă și memorare în cache
CNF-04	Caracter informativ	Nu sunt făcute tranzacții sau investiții
CNF-05	Portabilitate	Sistem complet containerizat Docker
CNF-06	Extensibilitate	Capabilități noi datorită MCP

3.2 Arhitectura Sistemului

Arhitectura este organizată pe straturi, asigurând o decuplare completă a tuturor componentelor pentru a preveni Single Points of Failure. În următoarea schemă este reprezentată această arhitectură.



Fluxul de Date: Utilizatorul în Flutter interoghează Backend-ul Asistentul AI, care apoi interoghează MCP Server pentru date live prin Python. Datele trec prin Circuit Breaker și LRU Cache, iar Rezultatul este livrat înapoi în frontend ca răspuns complet generat de Gemini 3.1 Flash Lite.

3.3 Proiectarea Fluxului de Date

Fluxul de date urmează un traseu clar printre straturile aplicației. Utilizatorul interacționează cu interfața Flutter, care trimite cereri HTTP/REST autentificate cu JWT către backend-ul FastAPI. Cererile care necesită date în timp real sunt delegate agentului AI, care interoghează serverul MCP pentru a apela uneltele Python necesare. Datele obținute de la sursele externe trec printr-un circuit breaker și prin memoria cache LRU, iar rezultatele finale sunt returnate în frontend ca răspunsuri complete generate de Gemini 3.1 Flash Lite.

În tabelul 3.3 listăm straturile aplicației și rolul fiecăruia în fluxul de date.

Table 3.3 - Straturile aplicației și responsabilitățile lor

Strat	Rol
Frontend (Flutter)	Interfața cu utilizatorul, gestionarea sesiunii JWT
Backend (FastAPI)	Rutare, validare Pydantic, coordonarea serviciilor
Agent (LangGraph)	Raționament ReAct, decizia apelurilor de unelte
Unelte (MCP)	Acces tipizat la date (portofoliu, piață, bancă)
Reziliență	Circuit Breaker + cache LRU între aplicație și sursele externe
Persistență (PostgreSQL)	Stocarea utilizatorilor, tranzacțiilor, istoricului

Ciclul de viață al unei cereri este ilustrat în figura 3.2. Astfel, trecem prin următoarele etape:

1. Frontend-ul Flutter trimite mesajul utilizatorului către `/api/v1/agent/chat`, atașând token-ul JWT.
2. Backend-ul reconstruiește istoricul conversației din PostgreSQL și încadrează mesajul între etichete `<USER_INPUT>` (protecție la injectare de prompt).
3. Agentul LangGraph decide, prin raționament ReAct, ce unelte MCP să apeleze.
4. Uneltele preiau datele de la sursele externe sau din baza de date, tecând prin Circuit Breaker și cache.
5. Modelul Gemini compune răspunsul final, care este salvat în istoric și returnat clientului.

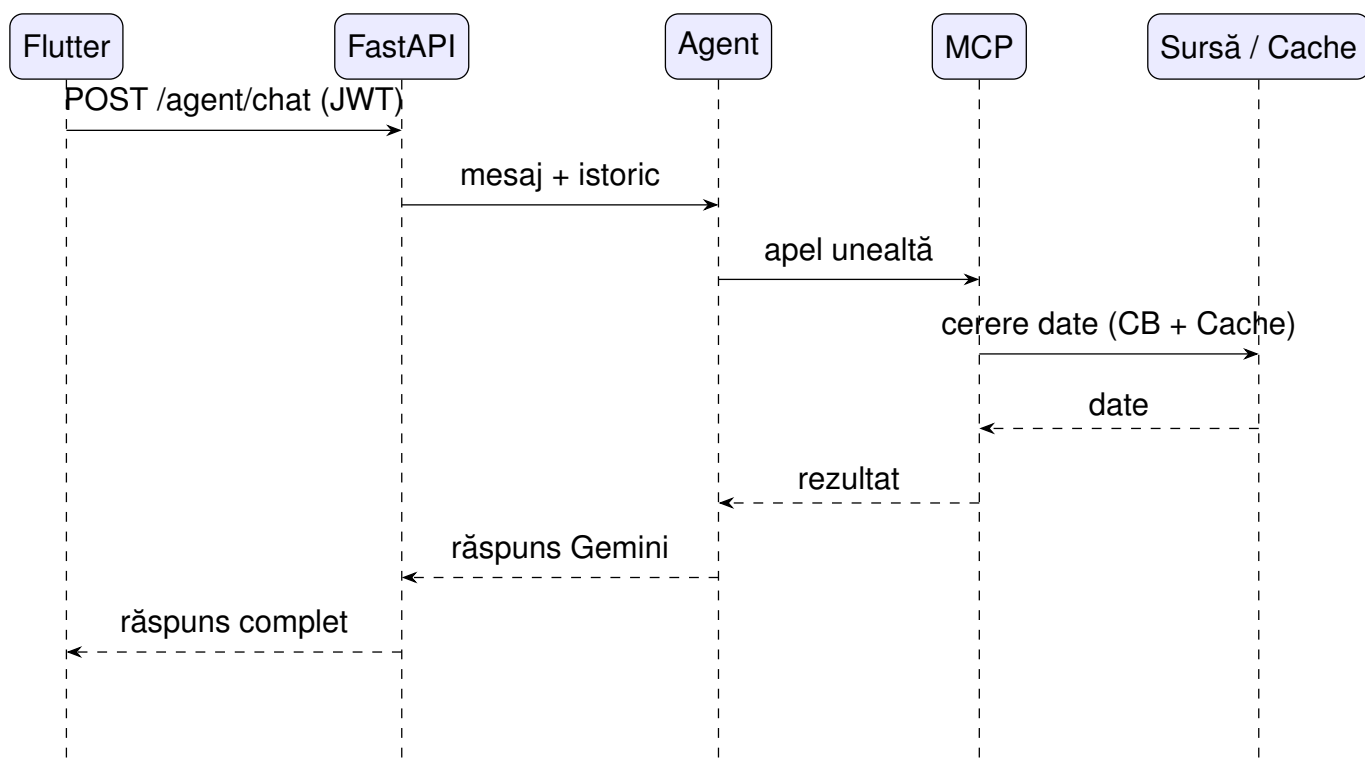


Figure 3.2 - Ciclul de viață al unei cereri prin straturile aplicației

4 IMPLEMENTAREA ALGORIMILOR DE DETECȚIE ȘI PREDICȚIE

Acest capitol prezintă partea tehnică a aplicației: algoritmi de detecție a anomaliilor și predicție a sănătății portofoliului, modul de voting și de predicție.

4.1 Ansamblul Hibrid pentru Detecția Anomaliilor

Pentru a monitoriza comportamentul portofoliului utilizatorului, și a detecta orice anomalii sau riscuri în evoluția acestuia, aplicația utilizează un ansamblu hibrid Ensemble Learning, fiind o tehnică de machine learning care combină predicțiile mai multor modele individuale pentru a obține un model final mai puternic, mai precis și mai robust, decât oricare din modele luate separat. Fiecare activ din portofoliu este transformat într-un vector de dimensiune fixă (maximum 10 poziții a câte 4 caracteristici: quantity, avgBuyPrice, currentValue, unrealizedPnL), completat cu valori nule pentru portofoliile mai mici. Antrenamentul se realizează pe date sintetice generate prin perturbarea acestor caracteristici cu zgomot aleatoriu de $\pm 5\%$.

Cele 3 Modele utilizate în ansamblu sunt:

- **1. Isolation Forest:** Utilizează implemterarea din biblioteca Scikit-learn, fiind un algoritm de detecție a anomaliilor care funcționează prin izolarea punctelor de date.
- **2. PCA Autoencoder:** Este un autoencoder, o rețea neuronală care învață să reproducă datele de intrare liniar prin descompunere Singular Value Decomposition, iar deviațiile bruște sunt semnalate de Mean Squared Error.
- **3. One-Class SVM:** Definieste o frontieră de decizie în spațiul caracteristicilor în jurul datelor normale, iar orice punct din afară este considerat anormal.

4.2 Mecanismul de Voting

Pentru mecanismul de voting este folosită tehnica Calibrated Soft Majority Voting. Fiecare model din ansamblu generează un scor de anomalie $S_i(x)$ normalizat între 0 și 1, unde valorile mai mari indică o probabilitate mai mare de a fi o anomalie. Decizia finală este luată pe baza unei medii ponderate a acestor scoruri, conform formulei 4.1:

$$S_{weighted} = w_1 * S_{IF}(x) + w_2 * S_{AE}(x) + w_3 * S_{SVM}(x) \quad (4.1)$$

unde w_i reprezintă greutățile atribuite fiecărui model, care sunt calibrate manual, iar dacă $S_{weighted}$ depășește un pragul de 0.5, atunci sistemul marchează portofoliul ca având o anomalie, cu nivele de încredere LOW/MEDIUM/HIGH corespunzătoare numărului de modele de la 0 la 3.

4.3 Modele de Predicție a Sănătății Portofoliului

Pentru prognoza sănătății portofoliului, este planificată integrarea a două modele:

- LSTM: Rețele neuronale recurente pentru serii temporale financiare cu dependențe pe termen lung. Avantajele principale includ capacitatea de a modela relații non-liniare complexe și sezonabilitate multiplă.
- Prophet: Un model statistic dezvoltat de Facebook pentru serii temporale cu tendințe clare și sezonabilitate anuală/săptămânală. Avantajul principal este robustețea la date lipsă și interpretabilitatea rezultatelor.

Pentru active volatile se recomandă utilizarea LSTM, în timp ce pentru active stabile se recomandă Prophet, cu un mecanism de selecție automat bazat pe volatilitatea istorică a activelor și a portofoliului.

5 IMPLEMENTAREA SOFTWARE A APLICAȚIEI

5.1 Tehnologii Utilizate

Aplicația e orientată pe execuție asincronă și containerizare, organizată pe patru module: back-end, inteligență artificială, securitate, și frontend. În figura 5.1 este prezentată arhitectura aplicației pe straturi.

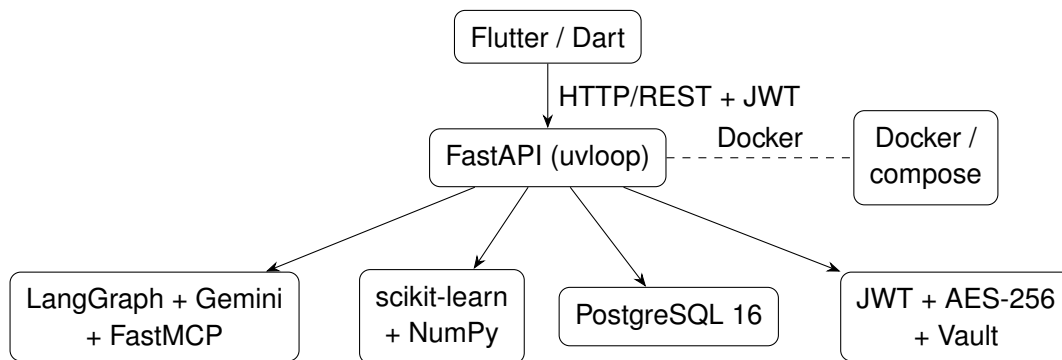


Figure 5.1 - Palierete tehnologice ale aplicației

Backend-ul este implementat în Python folosind framework-ul **FastAPI**, rulat de **uvicorn** peste bucla de evenimente **uvloop**. Accesul la baza de date se face asincron folosind **SQLAlchemy** și **asyncpg** pe baza de **PostgreSQL**. Migrările de schemă a bazei de date sunt gestionate de **Alembic**. Iar validarea datelor de intrare și ieșire se face cu **Pydantic**.

Inteligența artificială este reprezentată de agentul conversațional folosind **LangChain** și **LangGraph** pentru orchestrarea fluxului de date de tip ReAct. Modelul de limbaj folosit este **Google Gemini** (prin **langchain-google-genai**) și **FastMCP** pentru expunerea uneltelor. Componenta de învățare automată este implementată folosind **scikit-learn**, **NumPy**, **pandas** și **SciPy**.

Autentificarea și autorizarea se face folosind **python-jose** (JWT), parolele sunt hash-uite cu **bcrypt**, iar criptarea datelor se face folosind biblioteca **cryptography** (AES-256), cheia de criptare fiind stocată în **HashiCorp Vault** (client **hvac**).

Interfața utilizatorului este implementată în **Flutter** (limbajul **Dart**), cu un design responsive și temă întunecată Material 3. Întregul sistem este containerizat folosind **Docker** și orchestrarea se face cu **docker-compose**. Pornirea secvențială a aplicației fiind garantată prin verificări de sănătate (**pg_isready**).

În tabelul 5.1 sunt prezentate dependențele principale, versiunile și rolurile corespunzătoare.

Table 5.1 - Dependențe principale (backend), versiunile și rolurile lor

Bibliotecă	Versiune	RoI
fastapi	0.135.1	Framework web asincron
uvicorn / uvloop	0.41 / 0.22	Server ASGI și bucla de evenimente
SQLAlchemy	2.0.48	ORM asincron
asyncpg	0.31.0	Driver PostgreSQL asincron
alembic	1.18.4	Migrări de schemă
pydantic	2.12.5	Validarea datelor
langchain / langgraph	0.3+ / 1.0+	Orchestrarea agentului ReAct
langchain-google-genai	2.0+	Conector Google Gemini
mcp (FastMCP)	1.26+	Server de unelte MCP
scikit-learn	1.5.2	Modele ML (Isolation Forest, One-Class SVM)
numpy / pandas / scipy	1.26 / 2.2 / 1.13	Calcul numeric și date
python-jose	3.3.0	Token-uri JWT
passlib[bcrypt]	1.7.4	Hashing parole
cryptography	44.0.2	Criptare AES-256
hvac	2.1.0	Client HashiCorp Vault
httpx	0.28.1	Client HTTP asincron (eToro, BT)
yfinance	0.2.51	Cotații Yahoo Finance

5.2 Detalii de Implementare

Backend-ul FastAPI este organizate pe rute independente, sub prefixul `/api/v1`, fiecare rută delegând un serviciu dedicat (Tabelul 5.2). La pornire, mecanismul *lifespan hook* pre-înregistrează circuit breaker-urile, astfel încât acestea apar încă de la prima cerere în endpoint-ul de sănătate.

Table 5.2 - Routele aplicației

Rută	Responsabilitate
auth	Înregistrare, autentificare, emitere JWT
etoro	Portofoliul public de investiții
market	Cotații și istoric (Yahoo Finance)
anomaly	Detecția anomaliilor (ansamblu ML)
agent	Conversația cu asistentul Tori
bank	Integrarea bancară (BT PSD2)
budget	Gestionarea bugetelor
expenses	Analiza și categorizarea cheltuielilor
health	Starea sistemului (circuit breaker, cache, DB)

Persistența datelor este asigurată de PostgreSQL prin SQLAlchemy asincron. Baza de date conține nouă entități reprezentate în tabele (Tabelul 5.3), legate conform diagramei entitate-relație din Figura 5.2 (linie punctată = asociere logică prin `user_id`, fără constrângere de cheie străină).

Table 5.3 - Entitățile modelului de date

Tabel	Rol
users	Conturi de utilizator (parolă hash-uită, cheie eToro criptată)
chat_messages	Istoricul conversației cu Tori
assets	Instrumente financiare (simbol, nume, clasă)
portfolio_positions	Pozițiile din portofoliu
anomaly_logs	Rezultatele detecției de anomalii
bt_connections	Token-urile OAuth2 și consimțămintele BT
bank_transactions	Tranzacțiile bancare sincronizate
budgets	Bugetele lunare pe categorii
cache_entries	Cache persistent

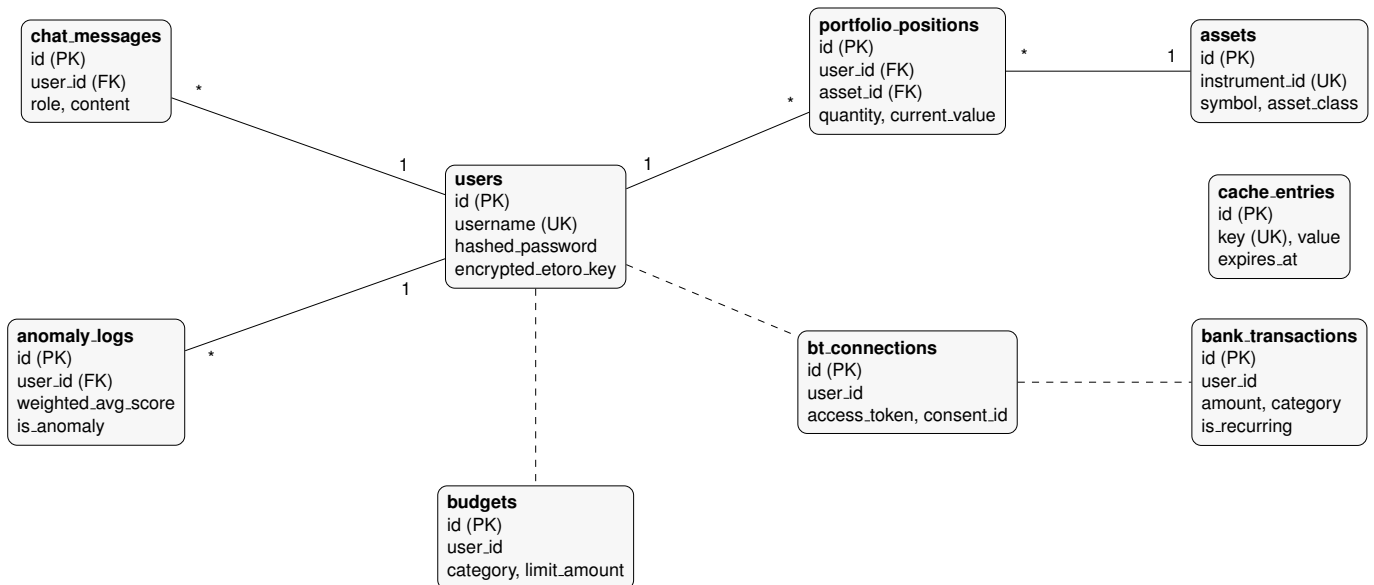


Figure 5.2 - Diagrama entitate-relație a modelului de date

Câmpurile sensibile sunt protejate prin criptare și decriptare la scriere și citire. (Secvența 5.1).

```

1 class User(Base):
2     __tablename__ = "users"
3     id: Mapped[int]
4     username: Mapped[str] # unic, indexat
5     hashed_password: Mapped[str] # hash bcrypt
6     encrypted_etoro_key: Mapped[str | None] # AES-256, nullable
7
8     @property
9     def etoro_key(self) -> str | None:
10         return decrypt_field(self.encrypted_etoro_key) if self.encrypted_etoro_key
11         ↪ else None
12
13     @etoro_key.setter
14     def etoro_key(self, value: str):
15         self.encrypted_etoro_key = encrypt_field(value)

```

Listing 5.1 - Model SQLAlchemy cu câmp criptat transparent

Asistentul Tori este un agent conversațional *LangGraph ReAct* pe baza Gemini. Uneltele agentului sunt expuse printr-un server **FastMCP**. Funcțiile Python sunt convertite în unelte LangChain (Secvența 5.2).

```

1 for tool in mcp_server._tool_manager.list_tools():
2     st = StructuredTool.from_function(
3         func=tool.fn, name=tool.name,
4         description=tool.description, coroutine=tool.fn,
5     )
6     tools.append(st)
7 agent = create_react_agent(llm, tools=tools)

```

Listing 5.2 - Convertirea uneltelor MCP în unelte LangChain

Avem opt unelte MCP (Tabelul 5.4): patru pentru investiții/piață și patru pentru datele bancare. Trebuie menționat faptul că *nicio unaltă nu are voie să arunce excepții*. O excepție ar întrerupe întregul ciclu al agentului. De aceea fiecare unaltă tratează erorile apărute și returnează un rezultat de tip `{"error": ...}` (Secvența 5.3).

Table 5.4 - Uneltele MCP disponibile agentului Tori

Unaltă	Rol
<code>get_my_portfolio</code>	Portofoliul live eToro
<code>get_all_instruments</code>	Lista instrumentelor
<code>get_stock_price</code>	Cotație în timp real (Yahoo Finance)
<code>get_market_sentiment</code>	Sentiment de piață (euristic)
<code>get_spending_summary</code>	Cheltuieli pe categorii / lună
<code>get_budget_status</code>	Cheltuieli vs. buget ("am depășit?")
<code>get_subscriptions</code>	Abonamentele recurente
<code>get_recent_transactions</code>	Ultimele tranzacții

```

1 @mcp_server.tool()
2 async def get_stock_price(symbol: str) -> dict:
3     """Obține cotația în timp real pentru un simbol."""
4     try:
5         return await market_service.get_stock_quote(symbol)
6     except Exception as e:
7         logger.warning(f"[MCP] get_stock_price({symbol}) esuat: {e}")
8         return {"error": "Cotația nu este disponibilă momentan."}

```

Listing 5.3 - Unaltă MCP rezistentă la erori

Mesajul utilizatorului este încadrat între etichetele `<USER_INPUT>...</USER_INPUT>`. Astfel, instrucțiunile care încearcă să suprascrie promptul de sistem sunt respinse pentru a proteja împotriva injectării de prompt. În absența unei chei `GOOGLE_API_KEY`, agentul revine la un model mock care informează utilizatorul de indisponibilitate.

Conectarea la Banca Transilvania urmează fluxul *OAuth2* cu extensia *PKCE*, ilustrat în Figura 5.3: la inițierea consimțământului se generează o pereche `code_verifier` / `code_challenge`, codul verifiier fiind stocat în baza de date, iar la callback, codul este schimbat cu token folosind verifiier-ul (Secvența 5.4).

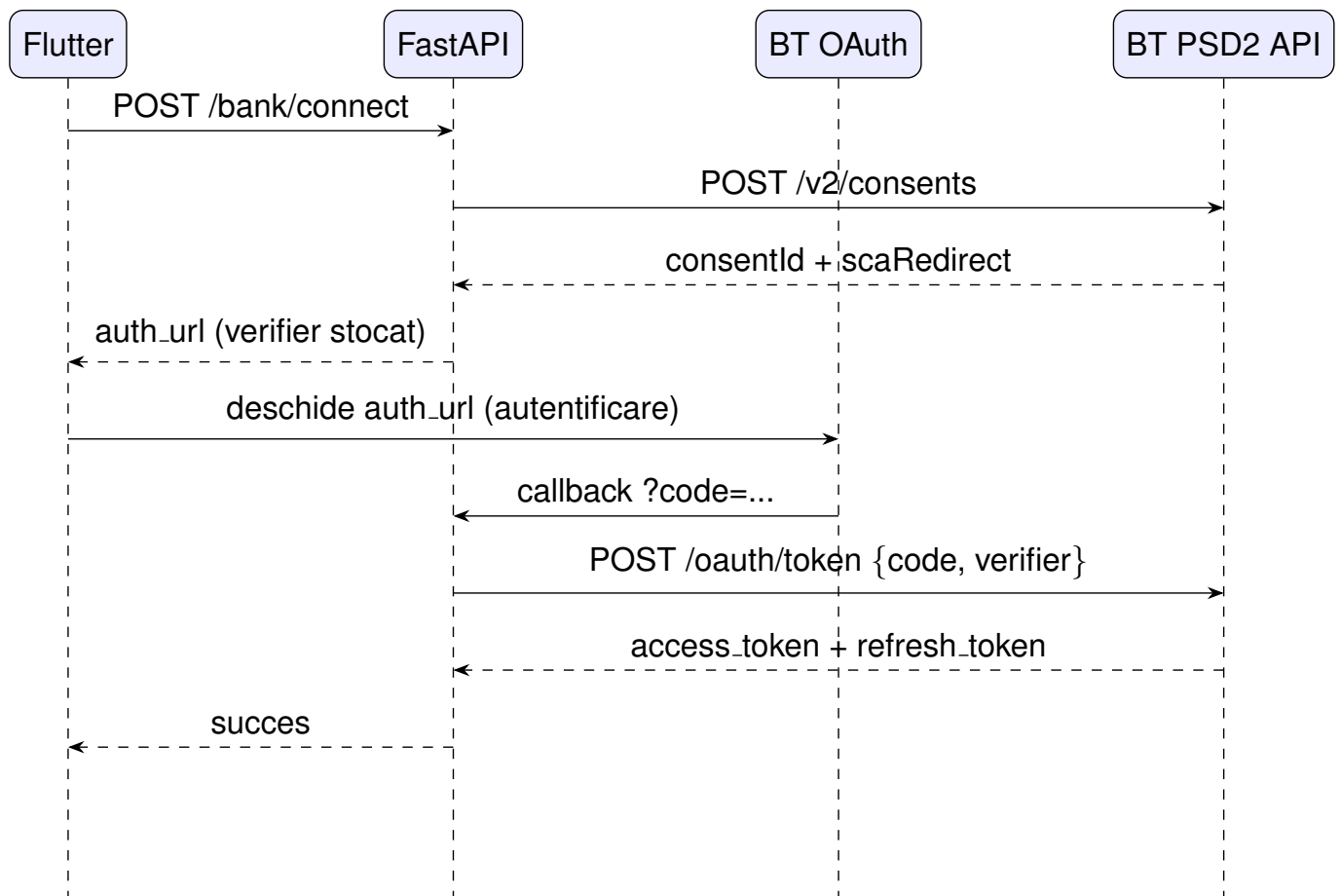


Figure 5.3 - Fluxul de autorizare OAuth2 PKCE (BT PSD2)

```

1 code_verifier = base64url(os.urandom(32)) # 256 biti aleatori
2 code_challenge = base64url(sha256(code_verifier)) # metoda S256
3 # URL-ul de autorizare include: ?code_challenge=...&code_challenge_method=S256
4 # Schimbul de token trimite: code_verifier=ORIGINAL

```

Listing 5.4 - Generarea perechii PKCE

În mediul de Sandbox, chiar dacă în exemple sunt folosite conturi de test, acestea nu pot fi rerulate (endpoint-ul `/v2/accounts` returnează HTTP 401 cu `accounts_count:0`). Astfel, aplicația folosește conturile exemplu din documentația oficială a BT (Tabelul 5.5), asigurând continuitatea aplicației. După autorizare, tranzacțiile a 120 zile sunt sincronizate și deduplicate prin `transaction_id`, apoi categorizate și analizate pentru a detecta abonamentele (Figura 5.4).

Table 5.5 - Conturi-exemplu folosite ca rezervă în sandbox

Cont	IBAN	Monedă
K13RONCRT0060214301	RO98BTRLRONCRT0ABCDEFGH	RON
K13EURCRT0060214301	RO98BTRLEURCRT0ABCDEFGH	EUR

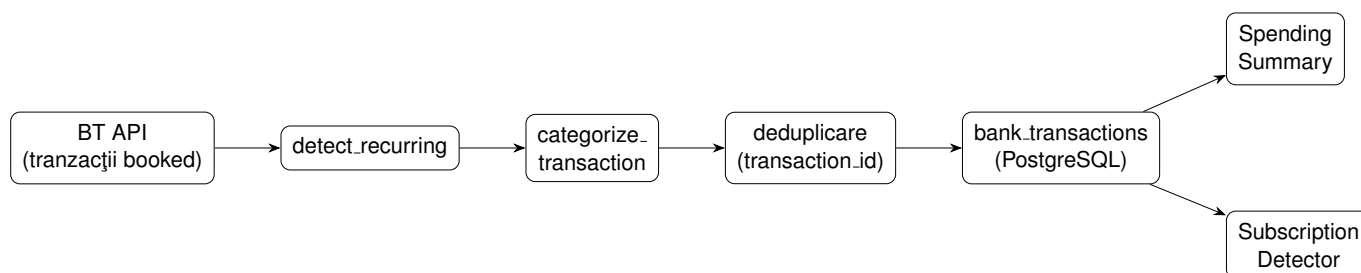


Figure 5.4 - Pipeline-ul de procesare a tranzacțiilor bancare

În timpul categorizării, fiecărei tranzacții i se atribuie o categorie prin aplicarea taxonomiei de cuvinte-cheie asupra comerciantului și a detaliilor tranzacției (Tabelul 5.6, Secvența 5.5). Pentru fiecare categorie se calculează cheltuielile lunare, se compară cu bugetele definite și identifică plățile recurente, în cazul unui pattern (Secvența 5.6).

Table 5.6 - Extras din taxonomia de categorii

Categorie	Cuvinte-cheie (exemple)
Food & Groceries	kaufland, lidl, carrefour, auchan, mega image, profi
Transport	omv, petrom, bolt, uber, taxi, metrou, cfr, wizz
Utilities	enel, digi, orange, vodafone, telekom, electrica
Dining	mcdonald, kfc, starbucks, restaurant, bistro, cafe
Subscriptions	netflix, spotify, adobe, microsoft 365, apple

```

1 def categorize_transaction(remittance_info: str, creditor_name: str) -> str:
2     text = f"{remittance_info or ''} {creditor_name or ''}".lower()
3     for category, keywords in _KEYWORD_MAP.items():
4         if any(kw in text for kw in keywords):
5             return category
6     return "Other"

```

Listing 5.5 - Categorizarea unei tranzacții

```

1 def detect_recurring(transactions: list[dict]) -> list[dict]:
2     # 1) tipar temporal: acelasi comerciant in 2+ luni, ~aceeasi zi (+/-3 zile)
3     recurring = set()
4     for cname, dates in creditor_dates.items():
5         if len(dates) >= 2:
6             days = [d.day for d in dates]
7             if len({d.month for d in dates}) >= 2 and (max(days) - min(days)) <= 3:
8                 recurring.add(cname)
9     # 2) decizie finala: potrivire cuvinte-cheie SAU tipar temporal
10    for tx in transactions:
11        text = f"{tx['creditorName']}"
12        ↪ {tx['remittanceInformationUnstructured']}".lower()
13        is_keyword = any(kw in text for kw in _SUBSCRIPTION_KEYWORDS)
14        tx["_isRecurring"] = is_keyword or (tx["creditorName"].lower() in recurring)
15    return transactions

```

Listing 5.6 - Detecția plăților recurente (cuvinte-cheie sau tipar temporal)

Parolele sunt stocate hash-uite cu bcrypt, autentificarea folosește token-uri JWT (HS256, valabile 24 h), iar câmpurile sensibile criptate cu AES-256 în mod CBC (Secvența 5.7). Cheia master de criptare este obținută din HashiCorp Vault, evitând expunerea ei. În Tabelul 5.7 sunt prezentate aceste mecanisme.

```

1 def encrypt_field(plaintext: str) -> str:
2     key = _get_aes_key() # 32 octeti din Vault
3     iv = os.urandom(16) # IV aleator de 128 biti
4     padder = padding.PKCS7(128).padder()
5     padded = padder.update(plaintext.encode()) + padder.finalize()
6     cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
7     ct = cipher.encryptor().update(padded) + cipher.encryptor().finalize()
8     return base64(json({"iv": base64(iv), "c": base64(ct)}))

```

Listing 5.7 - Criptarea unui câmp cu AES-256-CBC

Table 5.7 - Mecanisme de securitate

Mecanism	Bibliotecă	Scop
Hashing parole	passlib/bcrypt	Stocarea sigură a parolelor
Token-uri sesiune	python-jose (HS256)	Autentificarea cererilor
Criptare câmp	cryptography (AES-256-CBC)	Protejarea cheilor eToro
Gestiune secrete	hvac (Vault)	Cheia master AES
OAuth2 PKCE	implementare proprie	Autorizarea bancară

Reziliența se bazează pe *Circuit Breaker* asincron (deschidere după 5 eșecuri, revenire după 60 secunde, re-închidere după 2 succese) și pe o memorie cache pe 2 niveluri (LRU în memorie cu TTL și tabelul `cache_entries`). În Secvența 5.8 avem este configurat circuitul. Defectele și evaluarea scenariilor de eșec sunt prezentate în Capitolul 6.

```

1 @dataclass
2 class CircuitBreakerConfig:
3     failure_threshold: int = 5           # eșecuri înainte de deschidere
4     recovery_timeout: float = 60.0      # secunde până la testarea revenirii
5     success_threshold: int = 2          # succese necesare pentru re-închidere

```

Listing 5.8 - Configurația circuit breaker

5.3 Interfața Utilizatorului

Interfața este implementată în *Flutter*, valabil atât pe mobile cât și tabletă, desktop. Se folosește tema întunecată în stil *Material 3* și o paletă bazată pe culoarea de fundal #0D1117 și culoarea de accent #58A6FF. Codul este organizat pe directoare conform Secvenței 5.9.

```

1 flutter_app/lib/
2   main.dart           # punct de intrare + tema + restaurare sesiune
3   screens/            # ecranele aplicatiei (auth, dashboard, bank, ...)
4   services/api_service.dart # client HTTP + sesiune JWT
5   theme/app_colors.dart # paleta unica
6   utils/money.dart    # formatare RON/USD
7   models/             # modele de date
8   widgets/            # generative_ui, treemap_chart, empty_state

```

Listing 5.9 - Structura proiectului Flutter

Aplicația are 5 pagini principale, implementate cu `IndexedStack` pentru a păstra starea fiecărei pagini (poziția de derulare, istoricul conversației, textul introdus) atunci când are loc navigarea între pagini (Tabelul 5.8, Figura 5.5). Pe pagina de acasă avem și elemente de acces rapid către subpagini, navigarea fiind asigurată cu ajutorul `TabController`.

Table 5.8 - Filele principale ale aplicației

Pagină	Etichetă	Conținut
0	Acasă	Vedere de ansamblu: avere netă, acces rapid, investiții
1	Bancă	Conturi, solduri, tranzacții (BT PSD2)
2	Tori	Conversație (buton central evidențiat)
3	Bani	Sub-file: Buget, Abonamente, Analiză AI
4	Analiză	Sub-file: Grafice, Anomalii

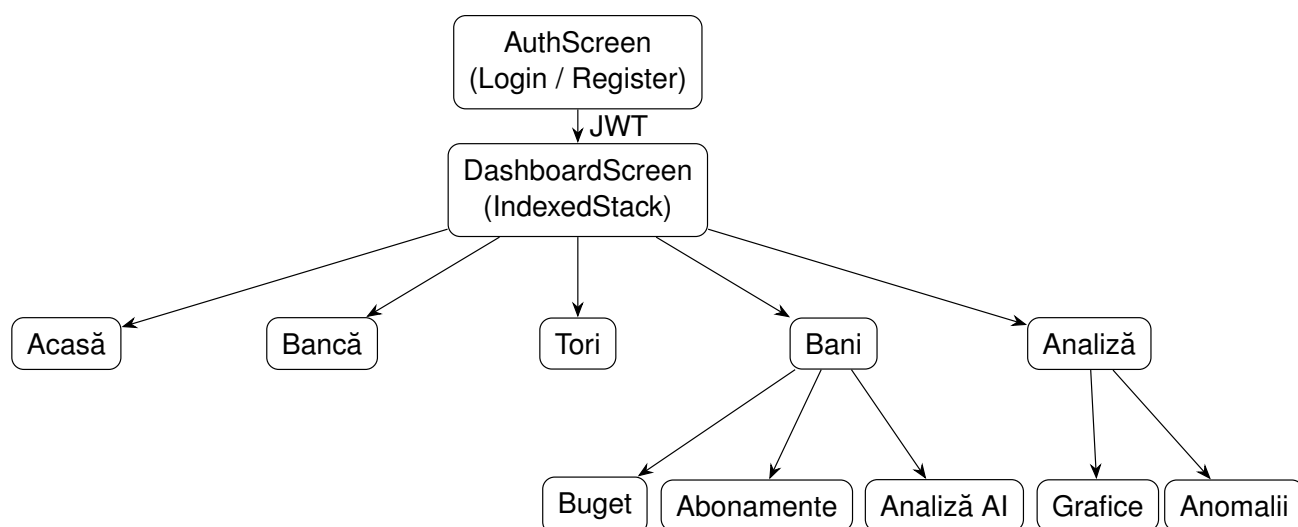


Figure 5.5 - Harta de navigare (IndexedStack cu cinci file)

Toate apelurile backend trec prin clientul HTTP `ApiService` care gestionează și sesiunea JWT, decând token-ul pentru a extrage identificatorul utilizatorului, îl persistă în `SharedPreferences` și îl restaurează la pornire (Secvența 5.10).

```

1 void _decodeToken(String token) {
2     final parts = token.split('.');
3     var payload = parts[1];
4     payload += '=' * ((4 - payload.length % 4) % 4); // padding base64
5     final map = jsonDecode(utf8.decode(base64Url.decode(payload)));
6     userId = int.tryParse(map['sub']?.toString() ?? '');
7     username = map['username']?.toString();
8 }

```

Listing 5.10 - Decodarea token-ului JWT în client

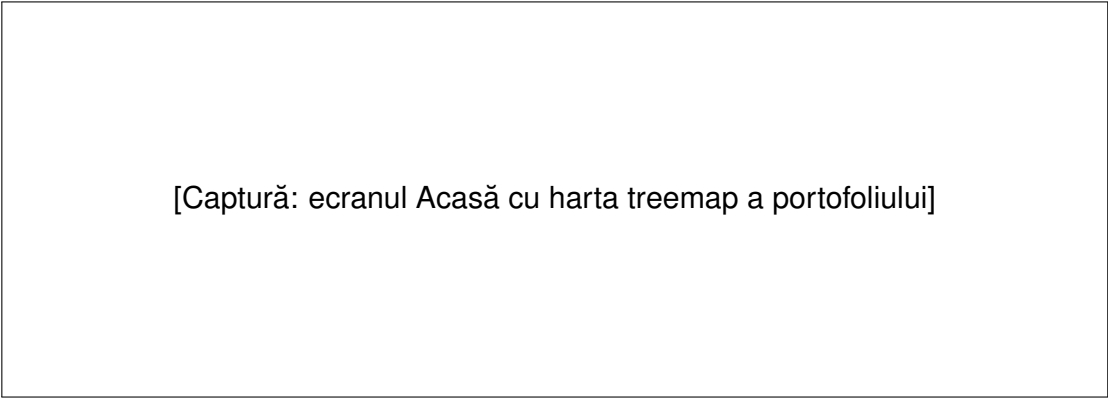
Răspunsurile lui Tori pot include componente interactive generate dinamic, prin blocuri widget direct în text (Secvența ??). La moment sunt trei tipuri: cursor de buget (*budget_slider*), chitanță

(*receipt*) și buton de acțiune (*action_button*). Butonul de execuție are stare proprie: în timpul execuției afișează un indicator de încărcare și raportează rezultatul final.

```
1 { "type": "budget_slider", "category": "Dining", "limit": 1000 }
2 { "type": "receipt", "merchant": "eMAG", "amount": 150.5, "date": "2026-06-05" }
3 { "type": "action_button", "label": "Sync Bank", "action": "sync_bank" }
```

Listing 5.11 - Componente generative emise de Tori

Pentru a vizualiza alocarea portofoliului și a cheltuielilor pe categorii se folosește un grafic de tip *treemap*. Listele goale folosesc o componentă comună de tip *empty state*, iar formatarea monetară este centralizată în utilita *Money*.



[Captură: ecranul Acasă cu harta treemap a portofoliului]

Figure 5.6 - Ecranul de ansamblu cu harta treemap a portofoliului

6 TESTARE, VALIDARE ȘI ANALIZA DEFECTELOR

În acest capitol sunt prezentate modurile de verificare a acurateții modelelor, clasificarea și lista defectelor identificate, și modul în care sistemul reacționează la datele eronate.

6.1 Strategia de Testare

Performanța asistentului financiar este măsurată folosind metrice standardizate, pentru cele două componente critice:

- **1. Natural Language Understanding:** Înțelegerea întrebărilor utilizatorului și acuratețea răspunsurilor generate de asistent. Aceasta poate fi evaluată prin scorul F1, care reprezintă media armonică între Precision și Recall, formula 6.1.

$$F1 = 2 * \frac{Precision \times Recall}{Precision + Recall} \quad (6.1)$$

- **2. Acuratețea Predicțiilor Financiare:** Pentru a evalua acuratețea predicțiilor financiare generate de asistent, utilizăm Mean Absolute Percentage Error, care măsoară eroarea procentuală medie între valorile prezise și cele reale.

6.2 Analiza și Documentarea Defectelor

Mecanismele de toleranță la defecte sunt esențiale pentru a asigura o experiență de utilizare fluidă. Sistemul de toleranță la defecte este implementat în mediul de execuție a aplicației, bazându-se pe cele trei surse de date externe: API-urile de la eToro, Yahoo Finance, și modelul LLM Google Gemini 3.1 Flash Lite. Fiecare dinte acestea reprezintă un potențial punct de eșec pe care aplicația nu îl poate controla, dar poate detecta și gestiona în mod corespunzător.

Principalul principiu arhitectural este Graceful Degradation: în loc să se blocheze, sistemul returnează un răspuns parțial sau o notificare de eroare, informând utilizatorul despre problema întâmpinată, dar în același timp permite utilizarea în continuare a aplicației.

Sistemul este proiectat să gestioneze defectele din tabelul 6.1 cu următoarele soluții:

Table 6.1 - Defecte, Mecanisme de Implementare și Soluții de Toleranță

Defect Potențial	Mecanism de Implementare	Soluție Tehnică
Downtime API eToro	Circuit Breaker (5 eșecuri)	Fallback la Date de Rezervă
Indisponibilitate Yahoo Finance	Circuit Breaker (5 eșecuri)	Fallback la cache LRU / date de rezervă
Inconsecvență Date	Validare prin scheme Pydantic	Respingere payload-uri
Latență Rețea	Timeouts asincrone (5-10s)	Întreruperea cererilor
Erori de Autentificare	JWT HS256 & bcrypt	Token-uri criptate și expirabile
Exfiltrarea Datelor	HashiCorp Vault	Cheile AES-256 sunt stocate în Vault
InstrumentId Necunoscut	Rezoluție dinamică	Instrumentele noi sunt identificate prin eToro

Clasificarea defectelor prezentată în tabelul 6.2 după tip, componentă, și impact:

Table 6.2 - Defecte, Clasificare și Impact

ID	Tip	Componentă	Probabilitate	Severitate
F-01	External Fault	eToro API	Medie	Critică
F-02	Indisponibilitate Sursă	Yahoo Finance API	Scăzută	Medie
F-03	Security Fault	eToro API Key	Scăzută	Critică
F-04	Data Integrity Error	InstrumentId Necunoscut	Medie	Medie
F-05	Infrastructure Fault	Postgres Down	Scăzută	Critică
F-06	Network Error	Latență Rețea	Medie	Medie
F-07	ML Drift	Modele ML	Ridică	Scăzută

Lista Defectelor Identificate și descrierea acestora:

- **F-01. Downtime API eToro:** API-ul returnează erori HTTP 500/503 sau nu răspunde. La 5 eșecuri consecutive, circuit breaker se activează, iar aplicația returnează date de rezervă.
- **F-02. Indisponibilitate Yahoo Finance:** Yahoo Finance nu necesită o cheie API și nu impune o limită zilnică fixă, dar poate limita cererile sau poate deveni temporar indisponibil. Apelurile sunt protejate de mecanismul Circuit Breaker, și atunci la eșecuri repetate sistemul servește ultima cotație din cache-ul LRU sau, în lipsa ei, date de rezervă.
- **F-03. Scurgere API eToro Key:** Cheile API pot fi exfiltrate, de aceea sunt stocate în HashiCorp Vault, criptate cu AES-256.
- **F-04. InstrumentId Necunoscut:** API-ul eToro returnează instrumentId-uri numerice, care nu sunt încă recunoscute de sistem. Atunci aceste id-uri sunt colectate și procesate prin alte API-uri eToro pentru a le identifica.
- **F-05. PostgreSQL Unavailable:** Stack-ul tehnologic poate iniția Backend-ul înaintea bazei de

date Postgres, ducând la erori de conexiune. Soluția este condiționarea pornirii Backend-ului prin dependență.

6.3 Evaluarea Rezilienței

Evaluarea rezilienței sistemului prin testarea scenariilor de defecte simulate:

- **Scenariul 1.** eToro API 100% indisponibil: Circuit breaker se activează după 5 eșecuri, iar aplicația returnează date de rezervă. Asistentul continuă să funcționeze în mod limitat bazându-se pe datele stocate anterior, dar nu poate oferi informații actualizate despre investiții, informând utilizatorul de această limită.
- **Scenariul 2.** Yahoo Finance API indisponibil: După 5 eșecuri consecutive, circuit breaker se activează, iar aplicația returnează date din cache, informând utilizatorul că datele de piață pot fi întârziate.
- **Scenariul 3.** HashiCorp Vault indisponibil: Sistemul detectează eroarea de autentificare și returnează un mesaj de eroare, informând utilizatorul că datele nu pot fi actualizate, dar permite în continuare utilizarea aplicației cu datele stocate anterior.

7 MANUAL DE UTILIZARE

In acest capitol vom trece pas cu pas fiecare functionalitate a aplicatiei.

8 CONCLUZII ȘI CONTRIBUȚII PERSONALE

Acest capitol sintetizează concluziile trase din dezvoltarea și testarea aplicației Virtual Personal Finance Advisor.

8.1 Concluzii

Această lucrare demonstrează necesitatea unui asistent financiar personal de informare, construit pe o arhitectură deschisă, orientată pe inteligența artificială. Aplicația *Virtual Personal Finance Advisor* integrează într-un singur sistem trei domenii: date de investiții, date bancare și analiză prin învățare automată, toate accesibile printr-o interfață conversațională în limbaj natural.

Spre deosebire de soluțiile existente, sistemul propus înlocuiește integrările rigide cu o arhitectură bazată pe *Model Context Protocol*, ceea ce permite extinderea facilă a capabilităților agentului fără modificarea modelului lingvistic. Decuplarea componentelor și mecanismele de toleranță la defecte asigură o experiență de utilizare neîntreruptă chiar și când unele surse externe sunt indisponibile temporar.

În tabelul 8.1 monitorizăm gradul de îndeplinire a obiectivelor propuse în Capitolul 1.

Table 8.1 - Gradul de îndeplinire a obiectivelor

Obiectiv	Stare
Urmărirea portofoliului de investiții	Realizat
Integrarea conturilor bancare	Realizat
Deteția anomaliilor prin ansamblu ML	Realizat
Asistent conversațional cu unelte MCP	Realizat
Toleranță la defecte și caching	Realizat
Predicția sănătății portofoliului	În dezvoltare
Autentificare aplicată per-utilizator	Planificat

În același timp, eliminăm riscurile legale și poziționăm sistemul și aplicația ca instrumente strict informative, educative și de suport, utilizatorul fiind în centrul deciziei.

8.2 Contribuții Personale

În Tabelul 8.2 sunt prezentate principalele contribuții personale și detaliate în continuare.

Table 8.2 - Contribuții personale și componentele asociate

Contribuție	Componentă
Integrarea portofoliului public eToro	services/etoro.py, instrument_resolver.py
Uneltele bancare ale agentului	mcp/server.py (4 unelte noi)
Stratul de reziliență	circuit_breaker.py, cache_service.py
Ansamblul de detecție a anomaliilor	ml/ (3 modele + vot)
Interfața generativă	widgets/generative_ui.dart
Containerizarea completă	Dockerfile, docker-compose.yaml

- **Integrarea portofoliului public eToro:** proiectarea unui model de alocare procentuală (raportat la o bază de referință) care transformă răspunsul endpoint-ului public într-o reprezentare coerentă a portofoliului, împreună cu un mecanism de rezoluție automată a instrumentelor necunoscute.
- **Uneltele bancare ale agentului (MCP):** extinderea agentului Tori cu unelte care citesc tranzacțiile, bugetele și abonamentele din baza de date, transformând capabilitățile declarate ale agentului în funcționalități reale (analiza cheltuielilor, verificarea depășirii bugetului), respectând principiul că nicio unaltă nu aruncă excepții.
- **Stratul de reziliență:** implementarea întrerupătoarelor de circuit, a memoriei cache pe două niveluri și a principiului de degradare grațioasă, astfel încât fiecare sursă externă să eșueze izolat, fără a afecta restul aplicației.
- **Ansamblul de detecție a anomaliilor:** combinarea a trei modele nesupervizate (Isolation Forest, autoencoder PCA, One-Class SVM) printr-un mecanism de vot ponderat calibrat.
- **Interfața generativă:** proiectarea componentelor interactive generate de agent direct în conversație (cursor de buget, chitanță, buton de acțiune), precum și migrarea sursei de date de piață de la Alpha Vantage la Yahoo Finance pentru a elimina limita zilnică de cereri.
- **Containerizarea completă:** orchestrarea întregului sistem prin Docker, cu pornire secvențială garantată și migrări de schemă automate.

8.3 Direcții Viitoare de Dezvoltare

Chiar dacă aplicația a se afla în curs de dezvoltare și multe funcționalități sunt deja implementate, există unele limitări și oportunități de îmbunătățire în viitor. Printre limite avem:

- Modelele ML pot suferi de ML Drift, ceea ce poate duce la scăderea acurateței predicțiilor în timp. O precizie mai mare necesită o acumulare continuă de date și reantrenare periodică a modelelor.
- Sursa de date de piață poate fi instabilă sau poate avea întârzieri, de aceea ca direcție vi-

itoare se are în vedere integrarea de surse de date alternative pentru a asigura continuitatea serviciului.

- Modelele de prognoză nu sunt încă integrate în aplicație, dar sunt în curs de dezvoltare.

Pentru a îmbunătăți aplicația în viitor, se planifică următoarele direcții de dezvoltare:

- Implementarea reantrenării automate a modelelor ML folosind pipeline-uri de date la fiecare minim 30 de snapshot-uri de date reale de portofoliu, pentru a menține acuratețea predicțiilor în timp.
- Extinderea MCP Server pentru analiza tehnică prin integrarea de indicatorilor pe date financiare istorice, cum ar fi Relative Strength Index pentru a măsura viteza și schimbarea mișcărilor de preț, Moving Average Convergence Divergence pentru identifica tendințele de piață, și Bollinger Bands pentru a analiza volatilitatea pieței.
- Implementarea unui sistem de notificări în timp real pentru a alerta utilizatorii de orice anomalii sau riscuri detectate de sistem.
- Îmbunătățirea autentificării și securității datelor utilizatorilor prin autentificare OAuth2.

